k-large-prime ladder: multi-target batch and triple large primes

Protocol `PROTOCOL.md`, frozen before data, with Amendments 1 and 2 (also frozen before their data). Every run's answer checked against the planted log; 0 failures in all cells. Goal (index calculus near rho parity) **not met**.

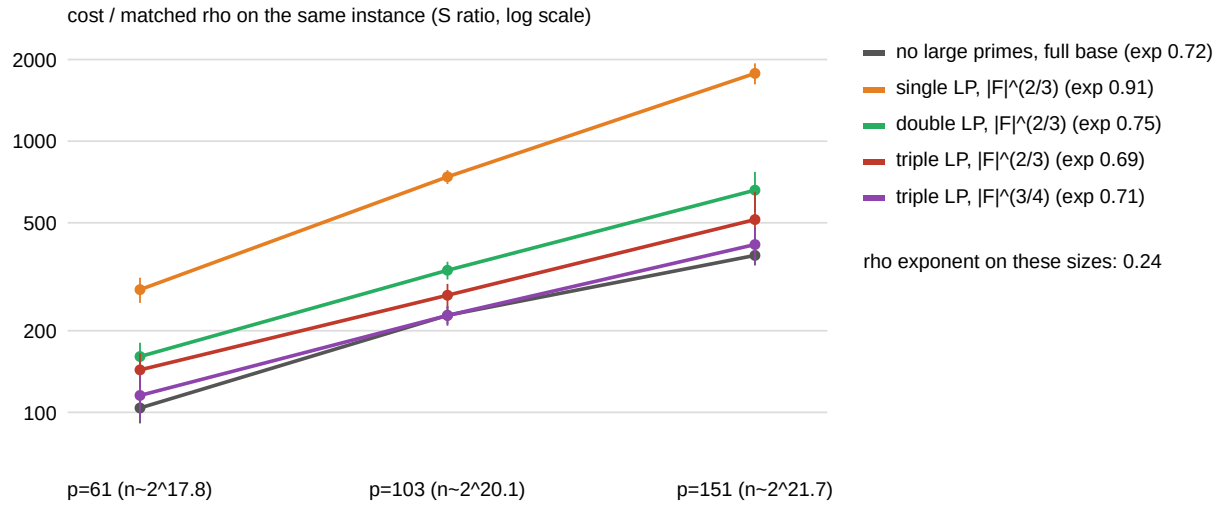
Amendment 1: multi-target batch (k = 1), prime-field curves 18/20/22 bits, 16 trials per cell

bits	T	B=0 cost / sqrt r per target	B=1 / B=0	gain vs T=1 (B=0)
18	1	1.922	1.278 ± 0.176	1.00
18	4	0.971	1.039 ± 0.094	1.98
18	16	0.615	0.945 ± 0.044	3.12
18	64	0.375	1.062 ± 0.028	5.12
20	1	1.846	1.119 ± 0.134	1.00
20	4	0.817	1.108 ± 0.100	2.26
20	16	0.535	1.033 ± 0.063	3.45
20	64	0.348	0.949 ± 0.039	5.30
22	1	1.580	1.314 ± 0.202	1.00
22	4	0.793	1.193 ± 0.081	1.99
22	16	0.502	1.046 ± 0.061	3.15
22	64	0.325	0.966 ± 0.051	4.86

Sharing one table across T = 64 targets cuts per-target cost 4.9-5.3x (ideal sqrt(64) = 8x). The residual collector's factor-base unknown (B = 1) costs 1.12-1.31x at T = 1 and 0.95-1.06x at T = 64: in a batch it reaches parity with multi-target rho (B = 0) and does not beat it. Comparisons are to multi-target rho at the same T, never to single-target rho.

Amendment 2: triple large primes, existing Gaudry engine on E/F_(p^3), 8 instances per size

Setting differs from the main protocol: a cubic extension field, where a subfield factor base exists (the prime-field k = 2..4 script could not be written in this session). Relations are 3-point decompositions, so quadruple large primes are not expressible in this engine. Cost is the engine's own S (field multiplications converted to additions), ratio to rho on the same instance.



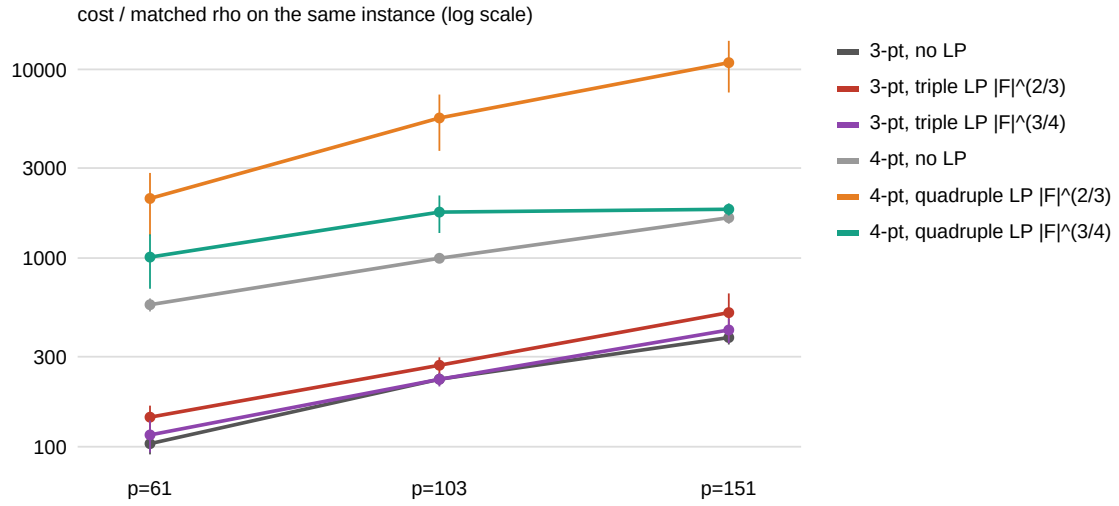
arm	p=61	p=103	p=151	exponent (log2 ops on log2 n)
no large primes, full base	104 ± 13	228 ± 16	379 ± 31	0.721
single LP, |F|^(2/3)	283 ± 30	739 ± 43	1778 ± 158	0.908
double LP, |F|^(2/3)	161 ± 20	334 ± 25	660 ± 111	0.750
triple LP, |F|^(2/3)	143 ± 22	270 ± 28	514 ± 136	0.687
triple LP, |F|^(3/4)	115 ± 22	228 ± 19	416 ± 62	0.715

rho	1	1	1	0.242

- **Triple vs single and double large primes** at the same small base |F|^(2/3): triple is the cheapest, 2.0-3.5x below single and 1.1-1.3x below double.
- **Triple vs no large primes**: the lowest IC exponent is triple at |F|^(2/3) (0.687, against 0.721 with no large primes), but at these sizes its cost is higher (513 vs 379 x rho at p = 151). Triple at |F|^(3/4) is within noise of no large primes at every size.
- **Against rho**: every IC arm is 100-1,800x rho, and the ratio grows with p. The rho exponent here (0.24) is far below 1/2 because fixed setup dominates rho at n of 2^18-2^22, so exponent comparisons with rho at these sizes are not meaningful; the ratio column is.

Amendment 3: quadruple large primes (engine extended to 4-point decompositions), same instances

`gaudry_cubic.rs` now has `arity = 4` (`subspace_quad_oracle`; default 3 unchanged), so a relation can carry four large primes; the existing eliminator removes them. 8 instances per size, 0 unsolved or incorrect. Decompositions with four large primes did occur (at |F|^(2/3): 178 / 625 / 1,368 summed over the 8 runs at p = 61 / 103 / 151).



arm	p=61	p=103	p=151	exponent
3-pt, no LP	104 ± 13	228 ± 16	379 ± 31	0.721
3-pt, triple LP |F|^(2/3)	143 ± 22	270 ± 28	514 ± 136	0.687
3-pt, triple LP |F|^(3/4)	115 ± 22	228 ± 19	416 ± 62	0.715
4-pt, no LP	566 ± 45	997 ± 48	1637 ± 116	0.615
4-pt, quadruple LP |F|^(2/3)	2069 ± 757	5528 ± 1828	10858 ± 3314	0.828
4-pt, quadruple LP |F|^(3/4)	1012 ± 324	1752 ± 394	1814 ± 134	0.479

- **Quadruple vs triple:** quadruple is 4-21x more expensive at the same base fraction (p = 151: 10,858 vs 514 at |F|^(2/3); 1,814 vs 416 at |F|^(3/4)). The 4-point oracle costs about 4|F|^2 pair tests a residual against 2|F| for 3 points, and the extra relations do not pay for it.
- **Exponents:** the 4-point arms fit lower slopes (0.615 with no LP, 0.479 for quadruple at |F|^(3/4)), but the |F|^(3/4) arm's cost at p = 151 is not above p = 103 within its error (1,752 ± 394 vs 1,814 ± 134), so a three-point fit on these sizes is not reliable, and rho's own slope here is 0.24 (setup-dominated). The ratio to rho grows with p in every arm.
- **Against rho:** every 4-point arm is 566-10,858x rho. Quadruple large primes do not move index calculus toward rho on these instances.

Amendments 4-5: batch parity with rho (22-bit prime curve, r = 2206429, 32 trials per cell)

Single-target parity is not reachable without relabelling: the collector needs B+1 collisions where rho needs one (floor 1.5x at B = 1), and a factor-base point with a known log turns it into rho itself. Parity was therefore tested in the batch, against multi-target rho (B = 0) at the same T. Frozen rule: the 95% interval of the B=1/B=0 per-target cost ratio contains 1 and lies inside [0.9, 1.1].

T	B=1 / B=0	95% interval	rule	batch cost vs single-target rho
64	0.974	[0.924, 1.024]	holds	4.9x cheaper

128	1.022	[0.971, 1.074]	holds	6.3x cheaper
256	0.935	[0.898, 0.971]	fails (low)	7.5x cheaper
256 (replication, fresh seeds)	1.015	[0.973, 1.057]	holds	7.8x cheaper

Reading (frozen in Amendment 5): the replication contains 1, so the first T = 256 cell is a chance deviation (one cell of four). **Parity with multi-target rho holds at T = 64, 128 and 256**. In the batch, the index-calculus structure costs nothing measurable over rho, and both are 4.9-7.8x cheaper per target than single-target rho. It does not beat multi-target rho. Scope: stdlib Python prototype, one curve, no negation map or distinguished points, group additions counted, every log verified.

Not done

- Prime-field k = 2, 3 (the main protocol): not run, for the same reason.
- Multi-target with k ≥ 2: not run.

Scripts: `scripts/mt.py`, `scripts/analyse_cubic.py`, `examples/klp_cubic.rs`; logs in `logs/`.